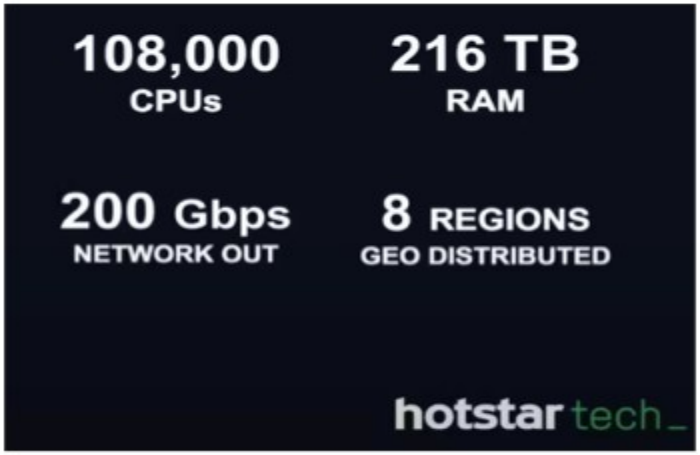
Container Orchestration

Priyanka Naik

JioHotstar

540+ crore views
Peak Concurrency of 6.12 crores

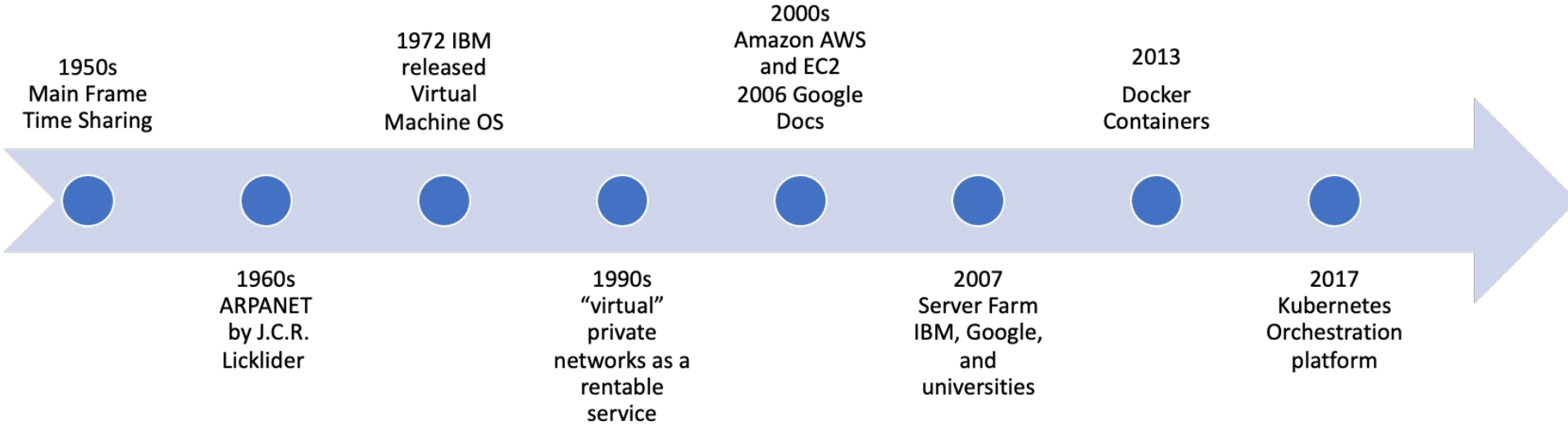
How did they scale?



Leveraging the cloud infrastructure

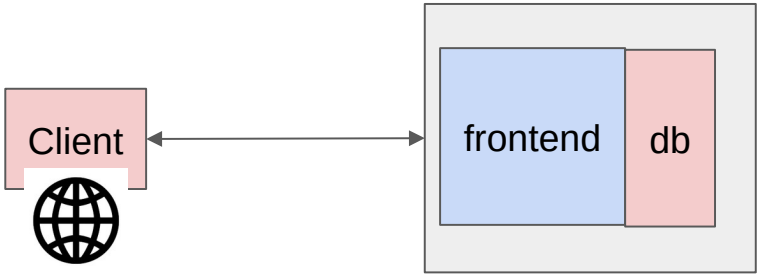
https://abhishekk.hashnode.dev/scaling-hotstar-for-millions-of-users-with-no-downtime-miracle-or-engineering

Cloud Evolution



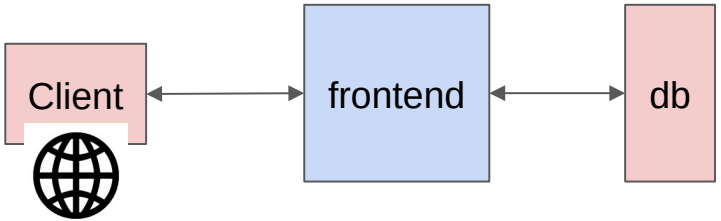
History of application deployment

Monolithic applications



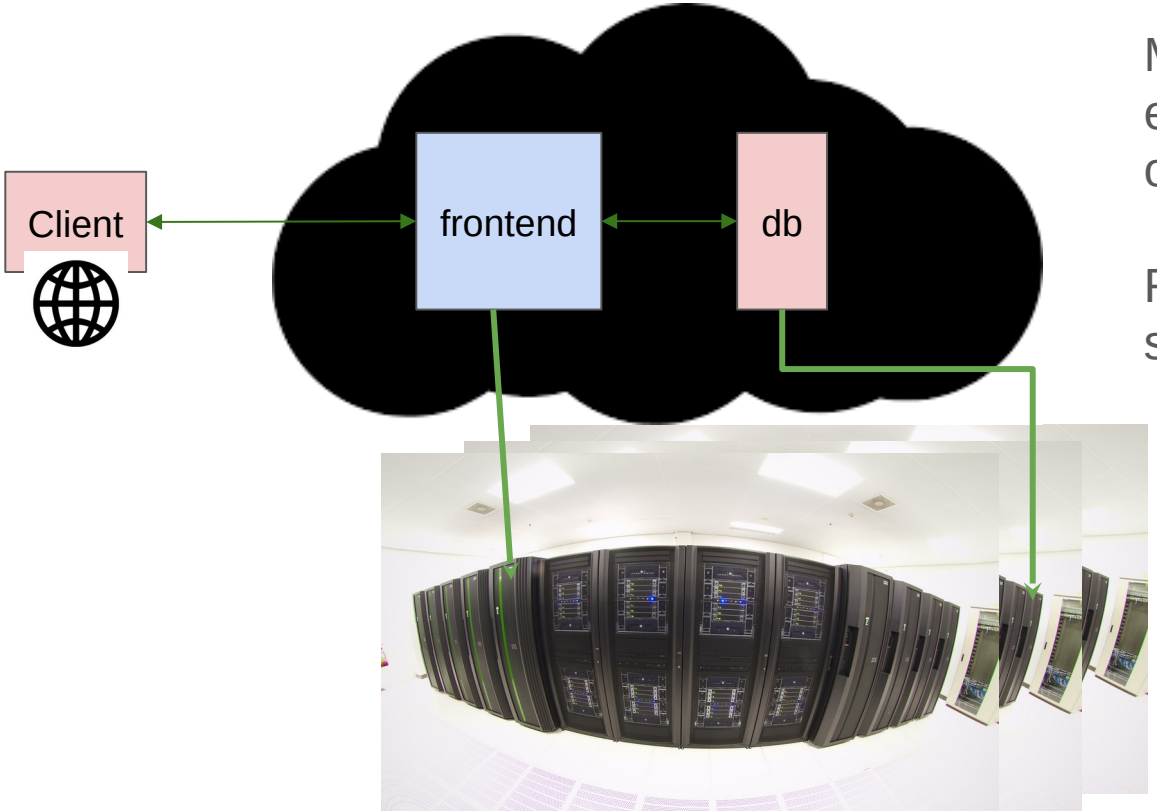
- Single executable
- Tight coupling
- Huge code base

Microservices



- Multiple small services
- Loose coupling
- Ease of managing/scaling individual service

Cloud deployments

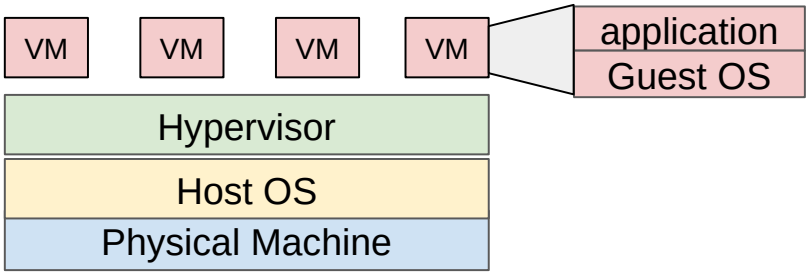


Microservice architecture enables easy deployment on cloud

Run large compute, storage task on the cloud

Virtualization

- Process Virtualization:
  - Run a process/application as is on different underlying machine architecture. E.g. Java Virtual Machine (JVM)
- **System Virtualization**:
  - Run a machine over a machine
  - Enables running multiple virtual machine on a single physical machine
  - Enabled through hypervisor/virtual machine monitor



Downsides of VMs

- Heavy-weight - need to run guest OS …
  - Consume a lot of resources to run
  - Are very large in size - typically in GBs
- Need something that is more lightweight that still gives us isolation and a reproducible environment

Containers

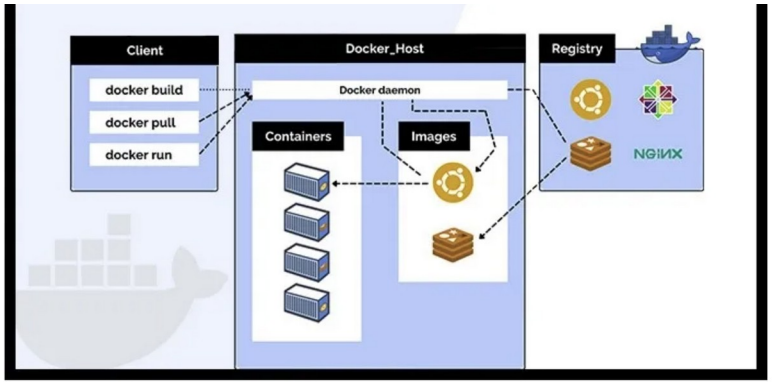
- Lightweight virtual machines without guest OS
- Isolation - no interference between 2 processes
- How does it achieve this:
    - Namespace: isolated view of resources like file system, network resources
    - cgroups: Resource limit per process/container

What is Docker

- Helps developer to build, deploy, update, and run containers.
- **Portability:** Saves time spent on setting up the environment and testing the application
- **Simple and Fast:** Users have to put their configuration into the code and deploy it.
- **Isolation:** Every Docker container has its resources isolated from other containers
- **Security:**control over container management and traffic flow

Docker Components

- **Docker Client**
  - Docker can connect to the host remotely and locally.
- **Docker Image**
  - Hold the entire metadata that elaborates the capabilities of the container.
  - Read-only binary templates in YAML.
- **Docker Daemon**
  - runs as a background process that manages parts like Docker networks, storage volumes, containers, and images.
  - start up command to client converts to HTTP API call to the daemon
  - only respond to the Docker API requests to perform the tasks.
- **Docker Networking**
  - Establishing communication between containers.
  - E.g. bridge
- **Docker Registry**
  - Location were docker images are stored
  - Docker Hub is the default storage location
- **Docker Container**
  - instance of an image that can be
  - created, started, moved, or deleted through a Docker API.



Demystify some questions

- Good alternative for Docker? Buildah, RunC, LXD, Podman, Kaniko
- Is it preferable to run more than one application in a single Docker container?
  - Experts always advise running a single application in a Docker container.
  - Attain better isolation, make it easier for building, testing, and scaling the application.
- What is the difference between Podman and Docker?
  - Docker uses Daemon as a component whereas Podman follows a daemonless architecture.
  - Podman needs another tool to manage services and support running containers in the background.
  - Systemd can also be integrated with Podman allowing it to run containers with systemd
  - Docker can build container images on its own. Podman requires the assistance of another tool called Buildah

Install docker

Prereq: Linux

docker -v

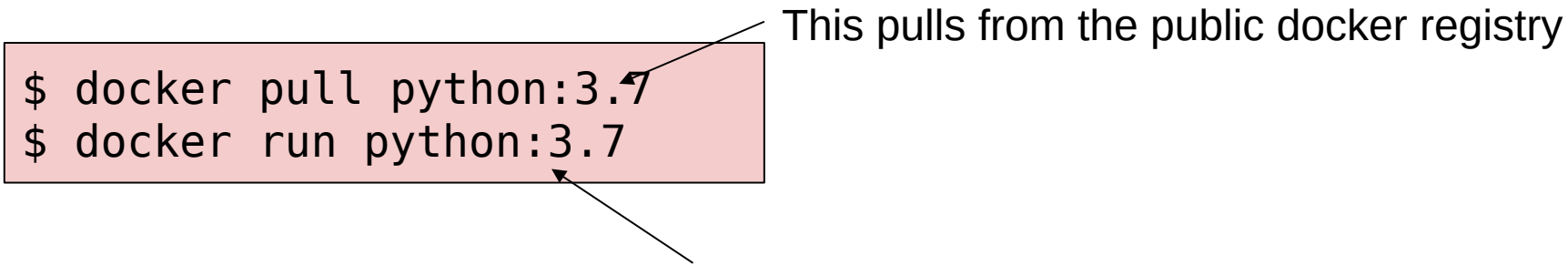
$ docker: command not found

https://docs.docker.com/engine/install/

How to run a container

Container image can either be built locally or “pulled” from a registry

Commands to pull a sample image and run it



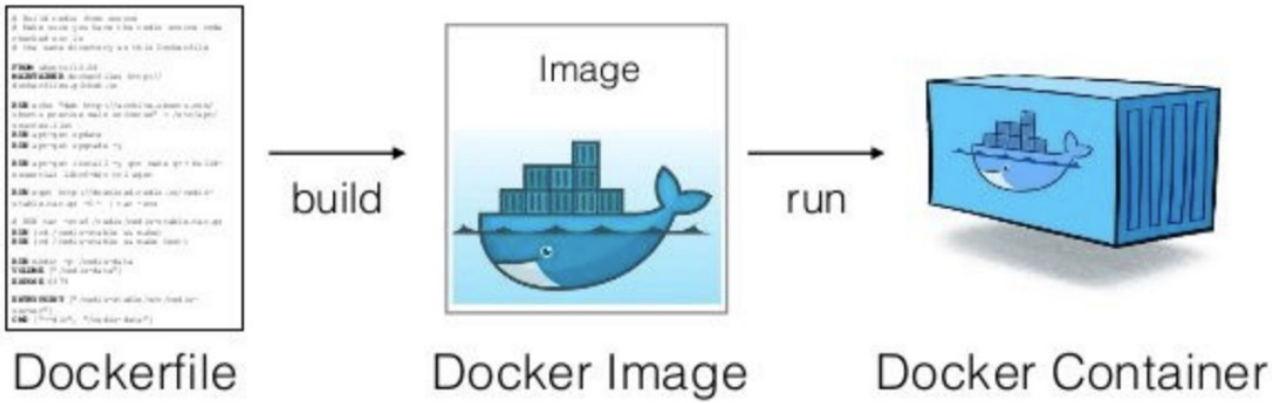


Image
build
run
Dockerfile
Docker Image
Docker Container

Featured solutions

Kubescape
Secure your Kubernetes cluster and gain insight into your cluster's security...
10K+

Ambassador Telepresence
Instantly bridge your workstation with Kubernetes clusters in the cloud. Test...
100K+

grafana/grafana
The official Grafana docker container
1B+

opensearchproject/open...
The Official Docker Image of OpenSearch (https://opensearch.org/)
10M+

Trending this week

homeassistant/amd64-a...
Mosquito add-on for Home Assistant.
5M+

paketobuildpacks/build
Cloud native buildpacks from the Paketo project.
50M+

vitess/lite
A slimmed down version of Vitess containers, with just the Vitess...
10M+

friendica
Welcome to the free social web.
58 1M+

New extensions

View all

Warp
Conveniently open your Docker containers in Warp terminal subshells.
100

Docker Labs K8s Toolkit
A Kubernetes debugging toolkit. Get your favorite shell with all your debug...
50

DockasaurusRX
Docker Resource Management Extension.
1K+

PHP Dumper
This extension allows you to call well-known 'dd' and 'dump' functions in you...
500

Databases

PostgreSQL
The PostgreSQL object-relational database system provides reliability a...
10K+ 1B+

MySQL
MySQL is an open-source relational database management system.
10K+ 1B+

Neo4j
Neo4j is a highly scalable, native graph database purpose-built to leverage not...
1.2K 100M+

MongoDB
MongoDB document databases provide high availability and easy scalability.
9.9K 1B+

AI and Machine Learning

tensorflow/tensorflow

pytorch/pytorch

langchain/langchain

ollama/ollama

grafana/grafana

Verified Publisher ☆

By Grafana Labs • Updated 2 days ago

The official Grafana docker container

Image

Pulls 1B+

Overview Tags

Grafana Docker image

Run the Grafana Docker container

Start the Docker container by binding Grafana to external port `3000`.

```
docker run -d --name=grafana -p 3000:3000 grafana/grafana
```

Try it out, default admin user credentials are admin/admin.

Further documentation on how to run Docker can be found at http://docs.grafana.org/installation/docker/.

Changelog

You can find information about Grafana in our docs, or find out what's new in the latest release here!

Docker Pull Command

```
docker pull grafana/grafana
```

TAG

10.1.5-ubuntu

Last pushed **2 months ago** by grafanaci

`docker pull grafana/grafana:10.1…`

DIGEST	OS/ARCH	COMPRESSED SIZE
5ac3f1301d65	linux/amd64	126.03 MB
300b576714ee	linux/arm64	120.62 MB

TAG

10.1.5

Last pushed **2 months ago** by grafanaci

`docker pull grafana/grafana:10.1…`

DIGEST	OS/ARCH	COMPRESSED SIZE
44021e67e80a	linux/amd64	104.51 MB
9403b0e3fb85	linux/arm64	96.76 MB

TAG

10.0.9-ubuntu

Last pushed **2 months ago** by grafanaci

`docker pull grafana/grafana:10.0…`

DIGEST	OS/ARCH	COMPRESSED SIZE
729288ab07b4	linux/amd64	112.75 MB
c92ea3d4c95c	linux/arm64	108.73 MB

TAG

10.0.9

Last pushed **2 months ago** by grafanaci

`docker pull grafana/grafana:10.0…`

DIGEST	OS/ARCH	COMPRESSED SIZE
dd8a9a766787	linux/amd64	90.37 MB
97da47db705d	linux/arm64	83.88 MB

TAG

9.5.13-ubuntu

Last pushed **2 months ago** by grafanaci

`docker pull grafana/grafana:9.5.…`

DIGEST	OS/ARCH	COMPRESSED SIZE

Let’s run a container

```
$ docker run nginx
```

Will pull the image and start running it

```
2023/12/11 13:30:57 [notice] 1#1: start worker process 94
```

```
$ docker ps
```

Lists all containers on the system

```
CONTAINER ID  IMAGE  COMMAND                  CREATED       STATUS       PORTS   NAMES
e036efae11db  nginx  "/docker-entrypoint."  3 min. ago   Up 3 min.   80/tcp
sweet_mccarthy
```

```
$ docker inspect <cid>/<name>
```

Very useful to examine the container and debug

Docker Image Size Investigation

If you see docker disk full issue you can use following command to debug

```
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" | sort -hr -k 3
```

In a image to check which layer is largest

```
docker history <imageid>
```

Delete all unused images

```
docker system prune -a
```

Publishing image

1. Log in (use a Quay robot account token, not your password)

docker login quay.io -u <quay-username>

Password prompt

2. Tag for Quay: quay.io/<namespace>/<repo>:<tag>

docker tag ml-app:v1 quay.io/myorg/ml-app:0.1.0

3. Push

d

Registry	Login Command	Image Path
Docker Hub	`docker login`	`docker.io/<user>/ml-app:0.1.0`
GitHub GHCR	`echo $PAT | docker login ghcr.io -u <user> --password-stdin`	`ghcr.io/<user>/ml-app:0.1.0`
AWS ECR	`aws ecr get-login-password | docker login --password-stdin <acct>.dkr.ecr.<region>.amazonaws.com`	`<acct>.dkr.ecr.<region>.amazonaws.com/ml-app:0.1.0`
GCP Artifact Registry	`gcloud auth configure-docker <region>-docker.pkg.dev`	`<region>-docker.pkg.dev/<project>/<repo>/ml-app:0.1.0`

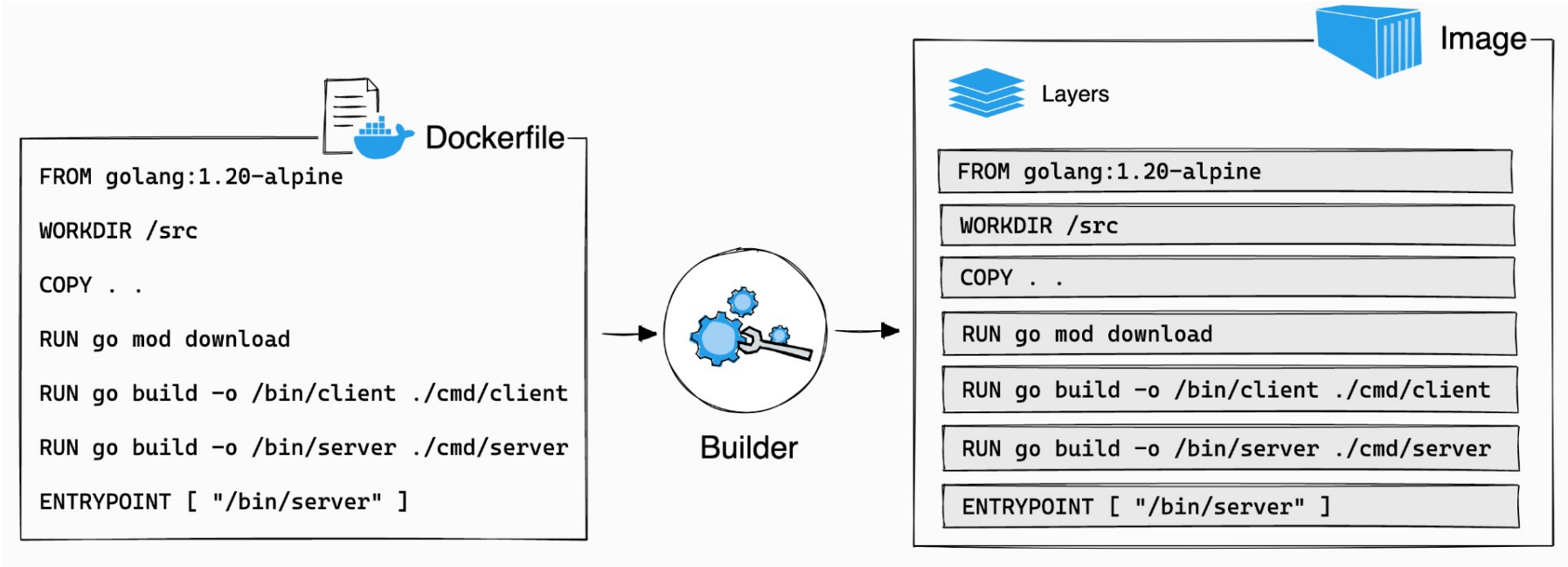
Docker compose

Allows to run multi-container applications - before kubernetes etc.

You specify multiple docker containers and it brings them all up.

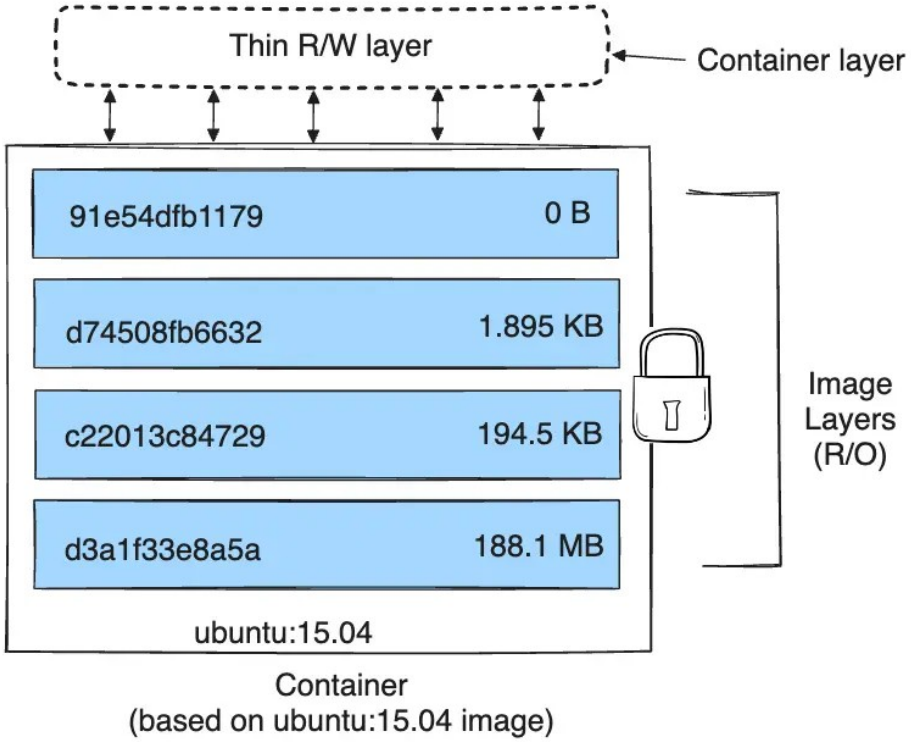
It sets up a single network for your entire application, all containers join them and can reach each other on this network.

Docker internals - layers



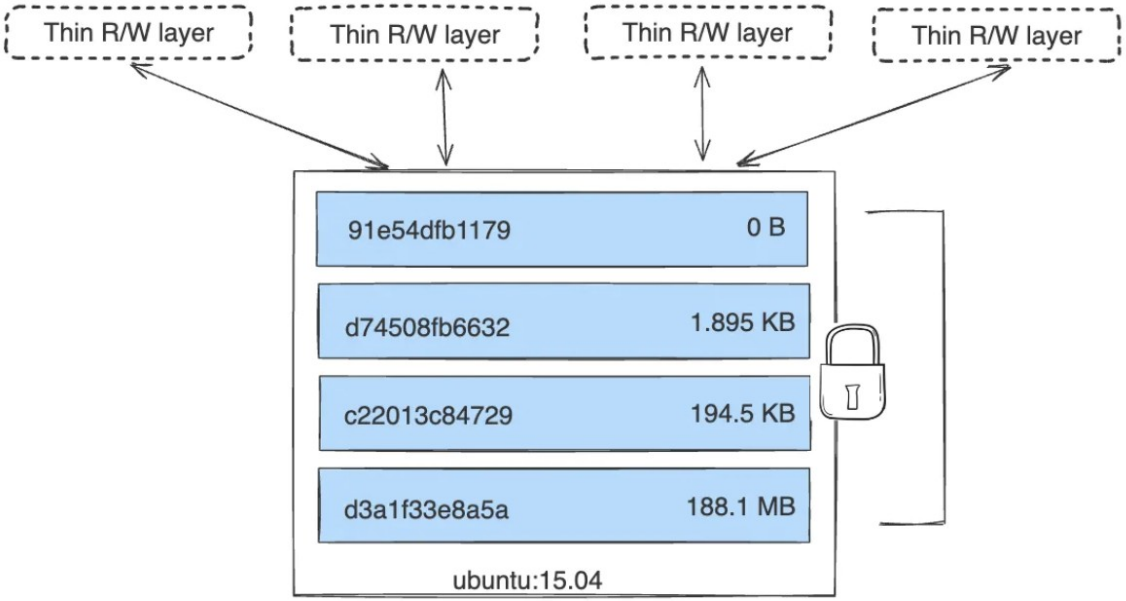
Layers

- Docker image is built as a series of layers, each layer represents a line in the Dockerfile.
- Every command that modifies the filesystem is a new layer.
- The layer only captures the diff from the previous layer.
- Layers are shared across different images.
- When we run a container, a new writable layer (called container layer) is created. Other layers are read-only.



https://docs.docker.com/storage/storagedriver/

Layers



https://docs.docker.com/storage/storagedriver/

ml-app-docker example

```
docker build -t ml-app:v1 .
docker run -d --name mycontainer1 -p 8000:8000 ml-app:v1

# In another terminal:
curl http://localhost:8000/health

curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"texts": ["I love this!", "This is awful."]}'

Expected response:
{"predictions":[
  {"text":"I love this!","label":"POSITIVE","score":0.9998},
  {"text":"This is awful.","label":"NEGATIVE","score":0.9997}
]}
```